

VULCAN: Instance-specialized, Verifiable Systems Heuristics Through LLM-driven Search

Anonymous Author(s)
Submission Id: 1508

Abstract

Systems resource management tasks rely primarily on hand-designed heuristics. However, growing hardware heterogeneity and workload diversity require heuristics specialized to particular deployment instances, making manual design expensive and difficult to scale. In this paper, we explore how to synthesize systems heuristics using LLMs. The main challenge is ensuring that generated heuristics execute safely, integrate correctly with the surrounding system, and still achieve strong performance. We propose VULCAN, a framework that identifies LLM-friendly interfaces that isolate core decision logic from the rest of the implementation. With VULCAN, LLM-generated code is restricted to simple stateless decision functions, while trusted runtime abstractions provide rich derived statistics for meaningful policy exploration without system-integration bugs. To ensure execution safety, LLMs synthesize heuristics in a restricted language, ANVIL, that guarantees important properties by construction. We evaluate VULCAN across three well-studied domains and demonstrate up to $4.9\times$ higher savings for spot-VM scheduling, up to $2\times$ lower miss ratios for cache eviction, and up to 10% higher application performance for tiered-memory systems, while ensuring execution safety throughout.

1 Introduction

Systems resource management has long relied on manually designed heuristics. This approach was sustainable when hardware was slower to evolve, deployment environments were more uniform, and a single carefully engineered policy could remain competitive across many settings. Today, systems operate across rapidly evolving hardware platforms, increasingly heterogeneous memory and compute hierarchies, and workloads that vary across tenants, applications, and time [59, 86, 94, 95, 107, 113]. As a result, the relevant unit of optimization is often an *instance*: a particular combination of hardware, workload mix, deployment setting, and optimization objective. The challenge is that designing and maintaining heuristics for each instance does not scale.

Neural models offered an early route to instance specialization [6, 22, 26, 27, 38, 61]. A learned policy can, in principle, capture complex dependencies in system state and adapt to a target environment better than a fixed hand-written rule. In practice, however, neural approaches remain difficult to deploy in core systems paths: their behavior is opaque, their training and serving pipelines add operational complexity, and integrating them with existing systems code

is cumbersome [31, 84]. More broadly, systems heuristics do not exist in isolation. They read state through existing interfaces, maintain auxiliary state over time, and rely on surrounding mechanisms to enact their decisions. Any automated approach must therefore satisfy a substantially higher bar than predictive quality alone: it must integrate correctly with the surrounding system and be safe to execute.

Recent advances in large language models suggest a different route. Unlike neural policies that must be embedded as external predictors, LLMs can synthesize *code*, enabling instance-specialized heuristics to be expressed as executable logic compiled directly into the host implementation. While enticing, this presents difficult challenges. If the LLM has unrestricted control over heuristic generation, it must reason not only about policy quality but also about state management, memory safety, API semantics, synchronization, and mechanism interaction (§2.2). Our experience, and that of recent work [24, 29], suggests that current LLMs can often discover performant heuristics under such freedom, but do not reliably produce code that satisfies the safety and integration requirements needed for deployment (§2.2.1). Conversely, constraining the LLM too narrowly preserves safety but sharply limits the policy space it can explore (§2.2.2).

This paper asks: *how should resource-management code be organized so that instance-specialized heuristic synthesis is practical?* Our answer is to refactor heuristics around a *synthesis boundary* that isolates core decision logic from the rest of the implementation. The key insight is that a small set of interfaces and runtime abstractions can make such synthesis both expressive and safe inside real systems. Concretely, we show that the core decision logic of a broad class of systems heuristics can be expressed through two interfaces, RANK and VALUE, and that constructing rich statistics for performant policies can be delegated to reusable feature-store abstractions. This organization permits the synthesis of verified policy code while keeping stateful logic and system-facing interaction in trusted, developer-managed code.

We realize this design in VULCAN, a framework for synthesizing instance-specialized heuristics as safe executable code. VULCAN rests on two key ideas. First, it decouples policy logic from state management. Developers retain responsibility for high-level task decomposition, policy triggers, raw feature collection, and the mechanisms that enact decisions; and LLM-synthesized artifacts are restricted to *stateless decision functions* behind RANK and VALUE interfaces

(§4.2). To enable meaningful policy exploration, VULCAN exposes rich derived statistics through libVulcan, a library of *listeners*: modular blocks that allow synthesized policies to use moving averages, percentiles, and other aggregates without implementing the data structures themselves (§4.3). Second, VULCAN synthesizes verified decision logic through ANVIL, a restricted domain-specific language (DSL) for LLM-generated code (§5). By excluding heap allocation, pointers, recursion, and unbounded loops, ANVIL makes important execution safety properties like memory safety, leak freedom, and termination, *provable by construction* for any well-formed program. Notably, the abstractions introduced by VULCAN allow ANVIL to be restrictive, yet produce powerful heuristics.

This design occupies a different point in the design space from both prior LLM-based heuristic synthesis and prior ML-for-systems work. Rather than asking the LLM to generate an entire heuristic implementation, VULCAN narrows synthesis to a structured policy layer that remains expressive enough for meaningful heuristic discovery. Rather than embedding an opaque learned model, VULCAN produces explicit policy code that integrates with existing implementations through narrow interfaces and trusted runtime support. We evaluate VULCAN across multiple resource-management settings: deadline-aware spot-VM scheduling, cache eviction, and memory tiering. Our findings show that VULCAN can synthesize both general-purpose and (when required) instance-specialized heuristics that are competitive with, and in several cases outperform, strong human-designed baselines, while satisfying execution-safety properties by construction.

In summary, we make the following contributions:

- We identify a refactoring of systems heuristics around a synthesis boundary and design a framework, VULCAN, that exposes only core decision logic to LLM-driven search while retaining system-facing structure in trusted code.
- We present libVulcan, that includes RANK/VALUE interfaces for decision logic and listener abstractions, which together provide a common substrate for expressive specialized systems heuristics.
- We introduce ANVIL, a restricted policy language that yields execution-safety properties by construction, enabling practical deployment of LLM-generated heuristics.
- We evaluate VULCAN for three well-studied domains: for spot VM scheduling, we find a *general* heuristic that outperforms all baselines to yield up to 4.9× savings; for the well-studied cache eviction problem, we find *specialized* heuristics that yield up to 2× improved miss ratios compared to baselines for specific instances; for tiered-memory systems, we find page promotion heuristics that yield up to 10% improvement in application performance; all while satisfying important execution safety properties.

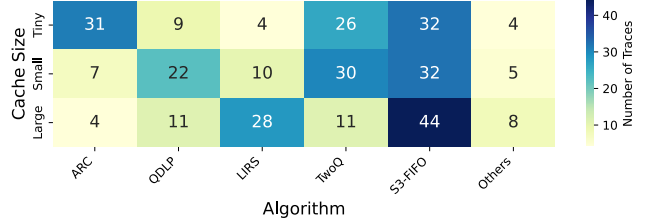


Figure 1. Count of CloudPhysics traces where each heuristic performs best (max. object hit rate). Tiny, small, and large imply cache sizes of 0.1%, 1%, and 10% of trace footprint.

2 The Pursuit of Instance-Specialization

Heuristics have been hand-crafted for various systems management tasks because optimal actions are often intractable to compute online. Action spaces for these tasks are large, decision intervals are fine-grained, and important system variables are often latent or partially observable. Consequently, practitioners design heuristics for specific *instances* of a problem, *i.e.*, particular combinations of workload, hardware, operating conditions, and optimization objectives such as performance, fairness, or utilization. Seen this way, systems research is often a search for *instance-specialized* heuristics rather than universally optimal ones.

2.1 An Example of Specialized Heuristics: Caching

We illustrate this search for specialized heuristics using the example of cache eviction policies. Different heuristics have been proposed for specific workloads, objectives, or deployment scenarios [16, 46, 63, 81, 93, 97, 108, 111]: some [63, 111] perform well for large cache sizes, while others [16, 46] are more suited for smaller caches. Workload characteristics also matter: scan-heavy workloads (mostly new objects) and churn-heavy workloads (mostly repeated objects) require different algorithms [81]. Heuristics have further been tailored for end-to-end *objectives* such as tail latency [18] and fairness [52], and for system-level *constraints* such as CPU overhead [93], lock-free design [108], and memory efficiency [30].

To quantify this observation, we executed 17 caching algorithms using libCacheSim [47] on 106 block I/O traces from the CloudPhysics dataset [98], where each trace represents a distinct instance (corresponding to a different tenant). Figure 1 shows our findings: no single algorithm performs best on even half the instances, and the set of competitive algorithms shifts significantly with cache size, *e.g.*, ARC is better for tiny caches while LIRS is better for large caches.

These findings are not limited to cache eviction: policies across the systems stack exhibit the same instance-dependence. Congestion control algorithms are tailored for Internet [5, 6, 12, 21] versus datacenter workloads [8, 51, 66], kernel queueing disciplines are tuned for varying performance and fairness objectives [33, 78, 87, 90], and cluster schedulers are fine-tuned for specific application classes such

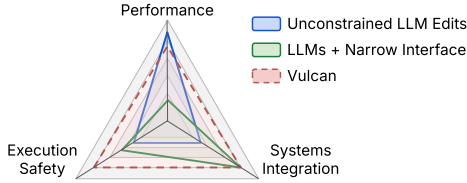


Figure 2. Illustration of three design alternatives. Unconstrained LLM edits result in execution safety and systems integration concerns (§2.2.1); overly constrained synthesis fails to achieve good performance (§2.2.2). VULCAN attempts to satisfy all three.

as data processing jobs [34, 35], cloud VM workloads [58], or deep learning training jobs [67, 102].

However, as workloads grow increasingly diverse, performance objectives become multi-dimensional, and hardware heterogeneity evolves rapidly, manually specializing heuristics for each instance is no longer tractable. This raises a natural question: *can heuristic specialization be automated?*

2.2 LLM-Based Synthesis: Requirements and Pitfalls

Recent advances in LLMs and coding agents, particularly in generating executable code [89, 99, 112], refining it using natural-language feedback [105], and carrying out multi-step reasoning [100], make automated heuristic specialization increasingly practical. Indeed, several recent systems [24, 29, 37, 48, 70] have already demonstrated that LLM-based approaches can discover effective resource-management heuristics for specific problems. While these advances are promising, building usable heuristics for practical systems imposes three key requirements.

- **Performance:** They must meet or exceed the performance of existing manually designed heuristics.
- **Execution safety:** They must be *proven* to be free from failures such as crashes, memory errors, leaks, or non-termination. Automated synthesis at scale makes the conventional practice of manual review impractical, so safety verification must be automated as well.
- **System integration:** Heuristics rarely run in isolation: they interact with other system components to read state and enact their decisions. Thus, the generated code must respect the APIs and semantics of the surrounding system.

Naively applying LLMs may not satisfy all three properties; below, we explore the design space to identify an approach that does. Figure 2 summarizes the alternatives, which we discuss in detail below.

2.2.1 Unconstrained Search Space Approaches. On one extreme of the design space are existing proposals for LLM-driven heuristic synthesis [24, 29, 37, 70]. These approaches place few restrictions on what the LLM generates: starting from decision logic, the LLM agent is free to allocate and manage state and interact with system APIs as it sees fit. This *unconstrained search space* maximizes the LLM’s

opportunity to discover high-performing heuristics, but also exposes the full implementation surface to errors.

We illustrate this using cache eviction as a running example. We compare two workflows: (i) an *evolutionary loop* [29, 82] that iteratively proposes, evaluates, and refines candidates; and (ii) an *agentic* workflow with access to codebase inspection and other tools. Both approaches used Claude Opus 4.5 [11] to synthesize heuristics for libCacheSim [47]. We evaluate the generated heuristics on ten traces from the CloudPhysics dataset [98]. Full details are provided in Appendix A; here, we discuss the key outcomes.

Evolutionary loop. Over fifty iterations, we found **system-integration violations**: several candidates (including the best-performing one) under-reported per-object metadata size, claiming only 16 bytes when they actually allocated up to 40 bytes. libCacheSim uses reported metadata size to check whether the cache has space for a new object; under-reporting causes more objects to be admitted than the cache capacity allows, artificially inflating the hit rate. We manually fixed these bugs and found that the resulting best heuristic achieved a 1.2% – 13.66% improvement over FIFO, which is comparable to several manual baselines.

Agentic workflow. We allowed a Claude Code agent [10] make edits to the codebase until a maximum cost budget is reached, and repeated this ten times independently. Across these ten heuristics, four modified internal libCacheSim counters that are read-only – another **system-integration violation**. One heuristic also introduced a *double-free bug*, an **execution safety violation**; incidentally, this specific bug was not triggered during testing, demonstrating that simple compile-and-test pipelines alone are insufficient. The improvements for the best heuristic (out of ten) were 0.2–23% over FIFO.

These results confirm that LLMs *can* discover performant heuristics, but do not reliably produce code that satisfies safety and integration requirements. Related work reports similar failures: ADRS [24] also noted similar use-after-erase and null dereference bugs, consistent with broader evidence that LLMs are less reliable on tasks requiring reasoning over complex codebases [44, 49, 73, 115].

2.2.2 Overly Restricted Search Space Approaches. The problems described in §2.2 arise because the LLM has unrestricted access to the implementation surface. The other extreme is to restrict LLM edits to narrow refinements of an existing heuristic [40], such that system integration and execution safety are preserved by construction. This includes tasks such as adjusting threshold values, adding conditionals, or reweighting existing features. However, shrinking the search space so aggressively forfeits the ability to discover substantially better policies.

To illustrate this, we revisit the evolutionary loop experiment for cache eviction but with a much narrower interface.

Inspired by HALP [93], we select eight eviction candidates using a *base heuristic* and then restrict the LLM-written function to choosing the final candidate from within that set. We supplement the LLM with several important features: count, size, last access, and insertion timestamps of each candidate, and percentile distributions of these statistics for all objects in cache. We experiment with both LRU and FIFO as base heuristics, running evolutionary search for 50 iterations each on the same traces as §2.2. We refer to the resulting best heuristics as Evolved-LRU and Evolved-FIFO, respectively.

Under this interface, the LLM cannot interact directly with libCacheSim, eliminating system-integration violations and substantially narrowing the space of execution-safety failures. Indeed, none of the synthesized heuristics exhibited execution-safety violations. However, the same restriction limits the search space too much to produce meaningful gains: the best Evolved-LRU heuristic improves MRR over FIFO by 0.05%–7.76%, nearly identical to LRU’s 0.04%–7.53%. Similarly, Evolved-FIFO improves over FIFO by only 0.01%–0.19%. These gains are far smaller than those in §2.2.

2.3 Towards A Better Division of Labor

While LLM-synthesized heuristics hold promise for instance-specialization, approaches at both extremes of the design space fail to meet all three requirements (Figure 2), due to a poor division of labor between the LLM and the developer. In the unconstrained setting (§2.2), the developer delegates everything to the LLM: decision logic, state management, and system interaction alike; in the over-restricted setting (§2.2.2), the developer retains so much control that the LLM cannot meaningfully explore. With VULCAN, we identify abstractions that enable a better division of labor: supporting meaningful policy exploration while preserving execution safety and system integration by construction.

3 VULCAN Overview

We present VULCAN, a framework for synthesizing safe, performant, and integration-compliant heuristics for systems resource-management tasks (see Figure 3).

3.1 Key Ideas

VULCAN rests on two key ideas that together address the three requirements from §2.2.

Idea 1: Abstractions to decouple core decision logic and state management (§4). To explore meaningful heuristics, LLMs must derive useful statistics from raw features and use them to produce a decision. VULCAN decouples these two steps and restricts LLM edits to: (i) generating code for stateless decision interfaces, and (ii) selecting and querying an API for rich derived features. We show that two stateless interfaces, RANK (ranking candidates) and VALUE (computing a scalar from system state), suffice to express the core decision logic of a broad class of resource-management heuristics

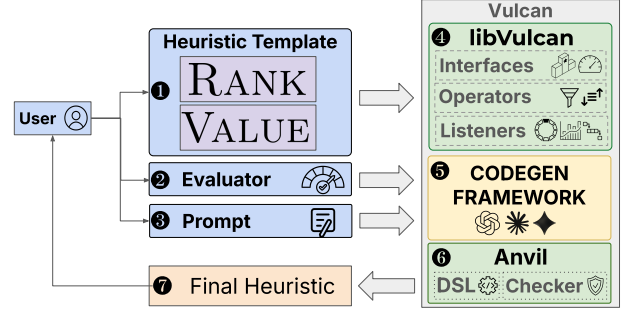


Figure 3. Overview of VULCAN. Users provide task-specific inputs – a template, a prompt, and an evaluator. VULCAN uses an code-gen framework to produce a safe and performant heuristic.

(§4.2). A library of *listeners* (§4.3) exposes useful statistics from raw features while handling the underlying state management. All other aspects of system integration remain the responsibility of developers.

Idea 2: Safety by construction via a restricted policy language (§5). As shown in §2.2, compile-and-evaluate pipelines alone are insufficient to ensure safety. VULCAN addresses this through ANVIL, a restricted DSL in which LLMs express decision logic. ANVIL omits heap allocation, pointer arithmetic, recursion, and unbounded loops; as a result memory safety, leak freedom, and termination are guaranteed by construction without the need for human review.

3.2 Workflow: Using VULCAN to Create Heuristics

Developers provide three inputs: a *heuristic template* (①), an *evaluator* (②), and a natural language prompt describing the resource-management task, the available raw features, and the evaluation methodology (③).

The heuristic template (based on Idea 1) exposes the core decision logic through RANK or VALUE interfaces and declares all raw features of interest. Developers define this template using libVulcan (④), VULCAN’s runtime library. The template and prompt are passed to a *code generation framework* (⑤), which synthesizes a heuristic in ANVIL (⑥) (based on Idea 2) by iteratively generating candidate heuristics, evaluating them on the user-provided evaluator, and using the feedback to improve. LLMs may also attach listeners to raw features, deriving statistics from simple means to arbitrary percentiles. The synthesized heuristic is passed to the evaluator, which scores it and returns feedback to the synthesis loop.

Once the search terminates, VULCAN returns the best-scoring candidate as the final heuristic (⑦). VULCAN is agnostic to the code-generation approach; in our case studies (§7), we use two approaches (ShinkaEvolve [54], OpenEvolve [88]) for our case studies.

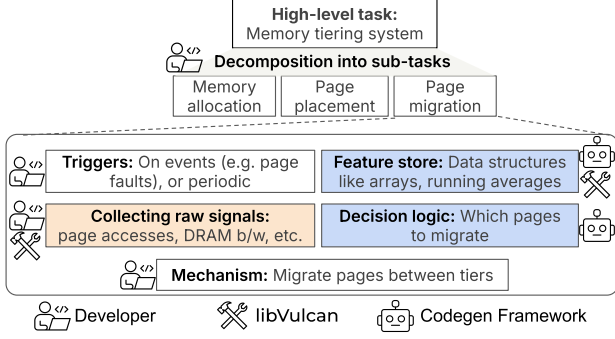


Figure 4. Typical components of systems heuristics (using example of memory tiering). VULCAN impacts the colored parts; icons show division between developers, libVulcan, and LLMs.

4 libVulcan: Interfaces for Policy Exploration

VULCAN narrows LLM synthesis to the *core decision logic* of a heuristic, delegating state management and system interaction to developer-owned code (§4.1). This division leads to two concrete abstractions: stateless decision interfaces (§4.2) and listener-based feature stores (§4.3).

4.1 Division of Labor in VULCAN

To motivate the division of labor in VULCAN, we examine the typical components of systems heuristics, illustrated in Figure 4 using a memory-tiering system.

A high-level task is typically *decomposed* into a collection of sub-policies; memory tiering, for example, uses separate policies for allocation [80], placement [62, 80], and migration [103] (Figure 4). Within each policy, there are five typical components. The *trigger* determines when the policy runs, e.g., page faults [80] or periodic scans [103]. Then, relevant statistics are *collected*: for the page migration policy in Figure 4, these include page accesses and DRAM bandwidth. Since processing raw signals at decision time can impose high overhead, a third component *stores* them in data structures that support efficient computation of temporal or statistical aggregates; for instance, a windowed average of DRAM bandwidth rather than per-timestamp measurements. The *decision logic* consumes these derived statistics to produce a policy output. Finally, *mechanisms* enact the decision, e.g., swapping pages between memory tiers.

With VULCAN, decomposition, triggers, and mechanisms (uncolored in Figure 4) remain the developer’s responsibility. These components require deep knowledge of system semantics: in the memory-tiering example, decomposition depends on how allocators and page fault handlers interact; adding new triggers may require changes to kernel page fault handling; and implementing migration requires correctly managing page metadata, locks, and reference counts. As §2.2 showed, even state-of-the-art LLMs do not reliably handle this complexity. VULCAN automates the remaining

components: feature storage and decision logic. Developers expose raw signals through a heuristic template (❶ in Figure 3), leaving open how those signals are aggregated and used in the decision. The code generation framework (❺) fills in this logic using libVulcan’s APIs (❹) for rich derived statistics.

Insight motivating libVulcan. libVulcan serves as the interface among developers, synthesized heuristics, and the system. A key observation is that many safety and integration bugs arise when LLMs are responsible for managing low-level feature state: allocating buffers, maintaining data structures, and updating them correctly across events. libVulcan eliminates this exposure by providing *stateless interfaces* for decision logic (§4.2) while encapsulating all state management in a *listener library* (§4.3).

4.2 VALUE and RANK Interfaces

The core decision logic of most resource-management heuristics takes one of two forms: it either *ranks* a set of candidates and selects a subset (e.g., evicting an object from cache), or it computes a scalar *value* that drives a resource-management action (e.g., computing a cwnd in congestion control). Table 1 shows that this pattern recurs across a broad class of systems tasks, and libVulcan exposes two corresponding interfaces, RANK and VALUE, for LLM-generated decision logic. When neither interface fits, it can be extended with a small number of additional functions.

Interface: VALUE. A VALUE heuristic is a function $\text{Value}(X) \rightarrow Y$, where X encodes system state as a set of features and Y is a scalar output corresponding to some control variable. Table 1 lists examples of typical VALUE tasks and their outputs. For instance, a congestion controller maps features such as `rtt` and `loss_rate` to a window size. To use the VALUE interface, developers define the function signature `double value_fn(const thor::store&)`, where `store` contains the state X and the return value is Y .

Interface: RANK. A RANK heuristic is a function $\text{Rank}(X, O) \rightarrow \mathcal{R}$, where $O = \{o_i \mapsto f_i\}$ maps N candidate objects to their features and $\mathcal{R} = [o_{i_1}, o_{i_2}, \dots]$ is an ordered list of all objects. The mechanism then selects from this ordered list as needed. Cache eviction, for example, ranks all objects in cache by their utility for retention (Table 1) and evicts the least-ranked object. As part of the template for RANK tasks, developers define the function signature for a per-object scoring function: `double score_fn(const thor::store&, int64_t obj_id)`. libVulcan provides utility functions to sample candidates and sort them by this `score_fn` in ascending or descending order as needed (§4.5).

4.3 Configurable Feature Stores Using Listeners

Stateless decision functions cannot compute derived features such as moving averages, percentiles, or population-level

Table 1. Examples of systems resource management tasks and the type of task (VALUE or RANK) they fall into.

Policy	Description	Type
Congestion control [36]	Decide number of unacked bytes in flight (cwnd).	VALUE: compute the cwnd
DVFS control [75]	Decide cpu_freq to balance performance & power.	VALUE: compute the frequency.
Cluster autoscaling [1]	Decide n_replicas to provision for a given service.	VALUE: compute replicas per service.
Hardware prefetching [65]	Choose an offset from the current access to prefetch data.	VALUE: compute the offset value.
Cache eviction [92]	Select which cached object(s) to evict.	RANK: all cached objects.
CPU scheduling [72]	Select which thread to schedule next.	RANK: all runnable threads.
Promotion in tiered-memory [80, 103]	Select which pages to promote to higher tier.	RANK: all memory pages.

Table 2. Listeners provided by libVulcan. The table shows the state each listener maintains, with update/query/space complexities. First six rows show listeners for temporal aggregates, while bottom two show listeners for aggregates over a population (across all objects).

Listener	Query API(s)	State and Data Structures	Complexity		
			Update	Query	Space
Average()	get_avg	sum, count	$O(1)$	$O(1)$	$O(1)$
MinMax()	get_min, get_max	min, max	$O(1)$	$O(1)$	$O(1)$
RollingWindow(N)	get_kth, get_avg	ring buffer, sum	$O(1)$	$O(1)$	$O(N)$
EWMA(α)	get_ewma	current EWMA value	$O(1)$	$O(1)$	$O(1)$
RollingPercentile(N)	get_percentile	ring buffer, ordered multiset [9]	$O(\log N)$	$O(\log N)$	$O(N)$
RollingCount(N)	get_count, contains	ring buffer, counts map	$O(1)$	$O(1)$	$O(N)$
PopPercentile()	get_pop_percentile	per-object latest, ordered multiset [9]	$O(\log M)$	$O(\log M)$	$O(M)$
PopMean()	get_pop_mean	per-object latest, sum	$O(1)$	$O(1)$	$O(M)$

statistics; yet smoothed averages [19, 20, 93, 103], summary statistics [21, 36], and histograms [16] are essential to strong heuristics. Delegating this state management to LLMs re-introduces the safety bugs from §2.2.

VULCAN resolves this tension through *listeners*: modular blocks that the LLM attaches to raw signals to create rich derived features. Once attached, their query APIs can be called inside the decision logic, whether a value function (VALUE) or a scoring function (RANK). Table 2 summarizes the listeners provided by libVulcan. Each listener tracks updates to its signal and maintains internal state for efficient queries. For example, an Average listener maintains running sum and count state and returns their ratio on query. libVulcan also supports more complex aggregates: temporal aggregates (e.g., RollingPercentile) and population-level aggregates over all candidate objects (e.g., PopMean). Temporal aggregates are useful for both RANK and VALUE tasks; population-level aggregates are primarily useful for RANK.

Listeners need only be reviewed and verified for execution safety once and a compact set of listeners can be reused across several systems tasks, as demonstrated by our case studies (§7). Listeners are also composable: multiple listeners can be attached to the same feature, and different features can use different listeners, forming a task-specific feature store without any LLM-written state management.

4.4 An Example of Using libVulcan

Listing 1 shows a cache eviction heuristic expressed through the RANK interface using libVulcan. The developer declares a feature store and its raw signals (Lines 2–4), and updates those signals as they arrive in on_insert and on_evict. The

Listing 1. Scaffolding for cache eviction expressed as a RANK task using libVulcan. LLM-generated code is highlighted in red.

```

1 void init():
2   vulcan::store store = {
3     .obj_signals: [{"insert_time",int}, {"access_time",int}]
4   };
5   store.add_listener("access_time", vulcan::Latest());
6   store.add_listener("insert_time", vulcan::PopPercentile());
7
8 void on_insert(ObjId o):
9   store.add_obj(o); store.update("insert_time", o, now());
10
11 void on_access(ObjId o):
12   store.update("access_time", o.id, now());
13
14 ObjId evict_obj():
15   R = vulcan::Sort(store.all_objs, score_fn, n_sample=32);
16   victim = R.back(); store.remove_obj(victim);
17   return victim;
18
19 float score_fn(vulcan::store& fs, int64 id):
20   auto recency = fs.get_latest("access_time", id)-now()
21   if fs.get_pop_percentile("insert_time", id) > 0.9:
22     return 6.0 * recency
23   else return 1.0 * recency

```

scaffolding samples 32 eviction candidates at random (Line 15), scores each via score_fn, and returns the lowest-scored object as the eviction victim (Line 17). The LLM specifies which listeners to attach to each feature (Lines 5–6) and implements the scoring logic (Lines 20–23) using the corresponding query APIs.

Table 3. Execution-safety properties guaranteed by ANVIL.

	Property
P1	Memory safety. No out-of-bounds access, use-after-free, or double-frees.
P2	Leak freedom. No unbounded memory growth over time.
P3	Termination. Must not run indefinitely; LLM scoring functions guaranteed to eventually terminate.

4.5 libVulcan Implementation

libVulcan implements the RANK and VALUE interfaces and all listener APIs as a C++ library. C/C++ systems can integrate libVulcan directly; Python-based systems use the same APIs via PyBind11 [45] bindings. Our case studies exercise both integration paths: spot VM scheduling (§7.1) uses Python bindings, while cache eviction (§7.2) and memory tiering (§7.3) use the C++ library.

For the RANK interface, libVulcan provides two sort strategies: OnDemandSort (used in §7.3), which scores and sorts candidates at decision time, and IncrementalSort (used in §7.2), which maintains candidates in a priority queue updated as features change, trading decision-time latency for incremental bookkeeping.

libVulcan totals ~3.5K lines of C++ code: ~1,300 for the core RANK and VALUE interfaces, ~1,000 for listeners, and ~250 for Python bindings.

5 ANVIL: DSL for LLM-written code

The abstractions in §4 reduce the surface area for execution-safety bugs, but LLMs can still generate code that leads to execution safety bugs. VULCAN closes this gap with ANVIL: a domain-specific language in which every well-formed program satisfies the execution-safety properties in Table 3 by construction.

Programming model. ANVIL is a restricted imperative language based on C++: it supports scalars, arithmetic, conditionals, bounded for loops, and built-in math functions (e.g., `anvil::max`), while excluding pointers, arrays, containers, heap allocation, recursion, and unbounded loops. These exclusions map directly to the execution safety properties (Table 3): no pointers or arrays preclude unsafe memory accesses (meeting P1); no heap allocation or persistent state preclude memory leaks and state-management errors (meeting P2); no recursion or unbounded loops preclude non-termination (meeting P3).

extern functions and trust model. ANVIL supports extern declarations for C/C++ functions; the compiler sees only a signature (name, scalar arguments, and a scalar return type), not the body. With ANVIL, safety extends transitively: a well-formed ANVIL program satisfies P1–P3 provided every linked extern does. In VULCAN, listener query APIs (Table 2) are exposed as externs, so synthesized heuristics inherit ANVIL’s guarantees relative to the correctness of libVulcan.

Compiling ANVIL code. ANVIL compiles in two stages: the frontend parses source against the ANVIL grammar and rejects disallowed constructs; the backend emits C, which is then compiled by standard toolchains (e.g., `gcc`). Because ANVIL syntax closely mirrors C, emitted code needs little translation: scalar expressions, conditionals, and bounded loops pass through unchanged, while ANVIL built-ins (e.g., `anvil::max`) resolve to equivalent C library calls.

Implementation. ANVIL is implemented in OCaml using Menhir [77] for parser generation, totaling approximately 3.5k lines of code: ~1,000 for the parser and lexer, ~1,500 for name resolution and class desugaring, and ~1,000 for the AST and C code generation.

6 Using VULCAN in Practice

Defining the target instance. A key requirement of VULCAN-generated heuristics is high performance, which often requires instance specialization (§2) across workloads, hardware configurations, operating regimes, or performance objectives. When instances differ enough that no single heuristic performs well across them (Figure 1), policies should be specialized to a particular instance, such as a specific cluster, tenant, workload class, or arrival pattern (as we do for cache eviction in §7.2). In contrast, for understudied problems where good solutions are not yet well-understood, even a single heuristic targeting average-case performance can yield substantial gains (as we do for spot VM scheduling in §7.1).

Cost–fidelity tradeoff in evaluators. VULCAN invokes the evaluator hundreds of times during search, so evaluation cost bounds the amount of exploration possible. Evaluating heuristics on real systems can take hours per candidate; in practice, we use cheaper proxies during the search phase: trace-driven simulation (§7.1, §7.2) or execution over a short representative workload (§7.3).

Natural language prompt. As discussed in §3, users must also provide a natural language prompt. Besides describing the task at hand, this prompt must also include information about libVulcan APIs, and (optionally) any relevant information about the evaluator, e.g., workload or instance information, instructions for heuristic design, etc.

Managing LLM API budgets. VULCAN’s narrow synthesis scope enables even mid-tier models to produce competitive heuristics, keeping API costs low. In our case studies, we use Claude Sonnet 4.5 for 80% of generations and invoke Claude Opus 4.5 intermittently to escape performance plateaus.

7 Case Studies and Results

We use VULCAN to synthesize heuristics for three resource-management tasks:

- **Spot VM scheduling** (§7.1): decide when a deadline-driven cloud job should use cheaper spot instances to minimize completion cost.
- **Web cache eviction policies** (§7.2): choose which cached objects to evict to maximize object hit rate.
- **Page promotion policy in tiered memory systems** (§7.3): choose which pages to promote from a slower memory tier to a smaller, faster tier.

The primary question we seek to answer through our evaluation is whether VULCAN can synthesize heuristics that outperform strong, human-written baselines across these diverse tasks. We also use individual case studies to answer supporting questions: (i) how VULCAN compares to unconstrained LLM-based synthesis, (ii) how its components (interfaces, listeners, and the DSL) affect synthesized heuristics, and (iii) the human effort and cost required to use VULCAN. Together, the case studies characterize the tradeoff between performance, safety, and synthesis effort.

7.1 Spot VM Scheduling

Cloud providers offer two pricing tiers: on-demand instances are reliably available but expensive, while spot instances are up to 11× cheaper [101] but can be preempted at any time. Deadline-sensitive jobs therefore typically avoid spot instances and pay the on-demand premium for predictability. Recent work [101] proposed a scheduler that cuts this cost by opportunistically switching between spot and on-demand while still meeting deadlines. We use VULCAN to synthesize scheduling heuristics for both the original single-region setting and a multi-region extension [24] where availability and prices vary across regions.

7.1.1 Single-region Scheduling. The scheduler is initialized with per-job constants: the required VM type, execution time, deadline, and the *changeover delay* incurred when switching a job between on-demand and spot instances. At each tick, the scheduler observes the current job status, progress so far, remaining work, and spot availability, then chooses one of three actions: run on spot, run on on-demand, or wait.

Template. Since the output of the heuristic at each tick must be a control variable that takes one of three values, this per-tick decision naturally follows the semantics of a VALUE task: that maps the observed state to a scalar corresponding to one of the three actions above. The per-tick features and per-job constants are passed as inputs to the VALUE function, and the LLM may attach listeners to the per-tick features.

Manually designed heuristics. We use two baselines from [101]. The first, a *greedy heuristic*, uses spot VMs opportunistically, waits when unavailable, and switches to on-demand only when further waiting risks missing the deadline. The second is *Uniform Progress* (UP), the state-of-the-art heuristic for this problem. Uniform Progress keeps the job on a linear progress trajectory from start to finish: it exploits

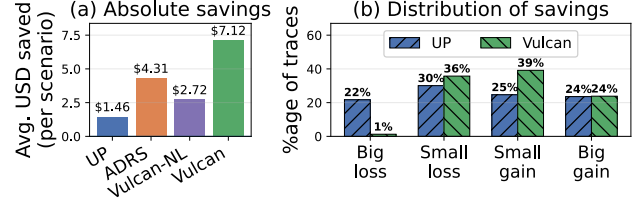


Figure 5. Comparison of schedulers for the single-region spot VM setting. (a) Mean average savings; higher is better. (b) big loss/gain = more than \$10, small = lesser than \$10

Listing 2. Regime detection in VULCAN’s scheduler.

```

1 double alpha = fs.get_avg(has_spot);
2 // longest consecutive spot availability streak (L_max)
3 int64_t L_max = 0, L_curr = 0;
4 for (int i = 0; i < 18; ++i):
5     if (fs.get_kth_recent(has_spot, i) == 1):
6         L_curr++; L_max = max(L_max, L_curr);
7     else L_curr = 0;
8 // regime detection: scarce when alpha low or small streaks
9 bool scarce = (alpha < 0.35) || (L_max < od_ticks && alpha < 0.5);

```

spot VMs when they are available, but if needed, switches back to on-demand to catch up with the linear pace.

Evaluator. We use the simulator and traces from [101]: spot-availability data for four AWS instance types (1- and 8-GPU K80/V100) paired with job configurations to yield 21,600 evaluation cases. During synthesis, candidates are scored by running a fixed 5% sample of all evaluation cases, and computing the *average USD savings* over the greedy baseline; all reported results use the full 21,600-case set.

Results. We run OpenEvolve [70, 82, 88] for 100 iterations and compare the resulting VULCAN scheduler against two baselines: (i) UP, the manual baseline above, and (ii) ADRS [24], which runs OpenEvolve with the same budget but allows unconstrained edits to the scheduler policy.

Main result. Figure 5a shows the mean savings of each scheduler over the greedy baseline. The VULCAN-synthesized scheduler saves nearly 4.9× more than Uniform Progress (UP), and 1.7× more than unconstrained synthesis (ADRS).

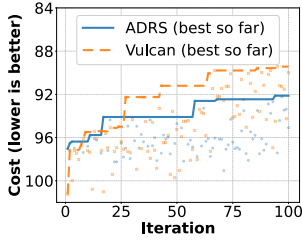
How VULCAN outperforms UP. The per-trace distribution (Figure 5b) reveals how VULCAN outperforms UP: UP performs worse than the greedy heuristic (*i.e.*, costs more money) on 52% of the evaluation cases. This pattern stems from UP’s core design assumption: it treats spot availability as *worst-case unreliable*. UP switches to on-demand whenever the job falls behind its expected progress schedule, *regardless of spot availability history*. In contrast, the VULCAN-synthesized heuristic (Listing 2) attaches RollingWindow(18) and Average listeners to track recent spot availability (has_spot), classifying each tick into a *scarce* or *abundant* regime and adapting accordingly: committing to on-demand sooner when spot looks unreliable, but staying patient and returning opportunistically when it does not.

Table 4. Summary of use cases, baselines, evaluation metrics, and evaluators.

Use Case	Baselines	Metric	Evaluator
Spot VM scheduling (single)	Greedy, Uniform Progress [101], ADRS [24]	Avg. USD saved	Simulator, 5% sample
Spot VM scheduling (multi)	Round-robin uniform progress [24]	Avg. USD saved	Simulator, 5% sample
Cache eviction	Multiple SoTA algorithms	Obj miss-rate reduction (MRR)	Simulator, 1M obj. sample
Memory tiering	ARMS [103]	App perf: goodput, latency	Emulator, one workload

VULCAN vs. unconstrained approaches. We observe that the performance of VULCAN Restricting synthesis to the core decision logic lets VULCAN improve rapidly (Figure 6), whereas the larger code surface of unconstrained search causes it to plateau.

Impact of removing listeners. Without listeners (VULCAN-NL), the heuristic is limited to the latest value of each feature. VULCAN-NL still outperforms UP, but performance degrades by roughly 2.4× compared to full VULCAN. This is because in the lack of listeners, rich derived statistics are unavailable – showcasing the benefits of listeners.

**Figure 6.** VULCAN vs. unconstrained search (ADRS).

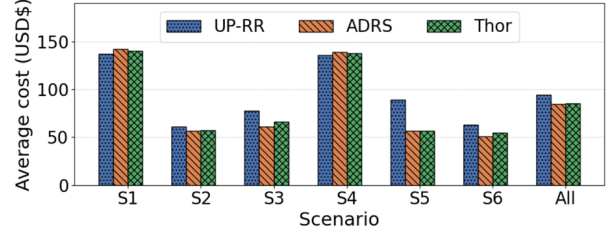
In contrast, each ADRS synthesis produces ~200 new lines that require manual review.

7.1.2 Extension to Multi-region Scheduling. We next extend the problem to multiple regions, where spot availability and prices vary across regions. The key constraint is that the scheduler observes availability only for its *current region*, requiring the heuristic to balance exploration and exploitation when choosing where to migrate.

Template. We decompose multi-region scheduling into two decisions. First, a *VALUE* function, invoked once per scheduler tick, chooses a high-level action: use spot or on-demand instances in the current region, or migrate to another region. Second, if migration is selected, a *RANK* function scores the candidate regions and selects the highest-ranked one. We formulate the second step as a *RANK* task (instead of *VALUE*) because regions may not have spot VM availability, so ranking an arbitrary set of candidates is more natural than predicting a fixed output value.

Evaluator. We use the multi-region simulator and benchmark from ADRS [24] that contains 600 total evaluation cases with AWS spot availability and prices for V100 GPU instances across nine U.S. regions/zones. During evolution,

Quantifying the developer effort with VULCAN. The heuristic template required 180 lines of code total: 152 for the template (110 C++, 42 Python) and 28 for the greedy seed. Beyond this one-time effort, ANVIL’s safety guarantees eliminate the need to manually verify any synthe-

**Figure 7.** Average per-scenario cost to run workloads. Lower is better.**Table 5.** Features provided to scoring function by our scaffolding.

Feature Type	Attributes
Per-object (f_i)	Access count, last access time, insertion time, size
Global (X)	Percentiles over access counts, ages, or sizes (e.g., p50 size of all cached objects). List of recently evicted objects (along with metadata) at eviction.

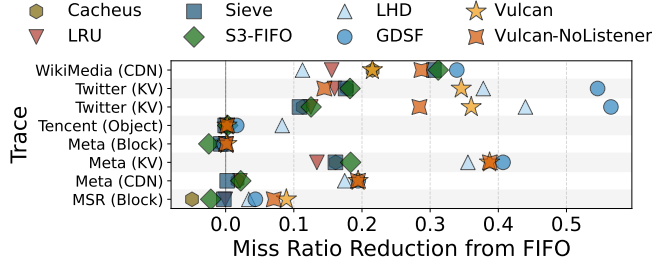
the evaluator runs each candidate heuristic on a fixed 5% sampled subset of all evaluation cases and returns the average cost per trace; after synthesis, the best policy is evaluated on the full benchmark.

Results. We again run OpenEvolve for 100 iterations and compare the result against: (i) UP-RR [24], a round-robin multi-region extension of UP, and (ii) an unconstrained-search baseline (ADRS). We report results for six scenarios [24] covering different combinations of availability zones and geographic regions, each with 100 evaluation cases. As Figure 7 shows, VULCAN outperforms UP-RR in four of six scenarios. VULCAN achieves only ~7% smaller savings than unconstrained search, demonstrating that its constrained interfaces can express performant heuristics while guaranteeing execution safety.

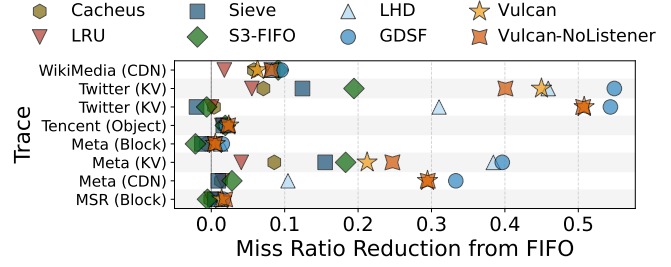
7.2 Cache Eviction

Cache eviction is central to performance in CDNs [13, 18, 42, 71, 92, 109] and in-memory stores [2, 3]. As §2.1 showed, access patterns and object characteristics vary substantially across deployments, making instance-specialized eviction heuristics valuable. Evicting an object from a pool of candidates maps naturally to a *RANK* task (§4.2): objects are scored by cache utility and the lowest-ranked object is evicted when space is needed. We evaluate whether VULCAN can synthesize such heuristics for individual cache deployments.

Template. We expose a *RANK* scoring function over a priority queue of cached objects; per-object metadata (age, size, last access time, recent eviction history; see Table 5) is



(a) Large cache size (10%), objects of different sizes



(b) Small cache size (0.1%), objects of different sizes

Figure 8. Performance of instance-specialized heuristics versus baselines for two classes of instances. Full results in Appendix B.

stored in a libVulcan feature store available to the function. To reduce overhead, we use a lazy update strategy: scores are recomputed only when an object is accessed, not on every cache event.

Evaluator. We use libcachesim [47] to simulate eight deployment-specific traces from Microsoft [68], Meta [17], Twitter [107], Tencent [113], and Wikimedia [4]. For each trace, we consider two cache sizes (10% and 0.1% of the trace footprint) and two object-size settings (uniform and variable), yielding 32 instances total. During synthesis, candidates are scored on the first 1M requests; the final heuristic is evaluated over the full trace, measuring miss ratio reduction (MRR) over FIFO (§2).

Results. We use seven state-of-the-art eviction algorithms as our baselines: GDSF [25], S3-FIFO [108], SIEVE [111], LHD [16], Cacheus [81], LRU, and FIFO. We use ShinkaEvolue [54] as the code-generation framework to generate heuristics for each of the 32 instances described above. We also run an ablation where we disable listeners, and only provide simple statistics (named VULCAN-NoListener).

Overall results. Across the 32 instances, VULCAN-generated heuristics outperform *all* baselines on 6 instances and are within 5% of the best on 13 more; 4 of the remaining 13 are within 10% of the best. All results are achieved automatically with a unified implementation requiring only a one-time development effort.

Breakdown of results. Figures 8a and 8b show miss-ratio improvements over FIFO for two instance classes. For the 10% cache configuration, VULCAN outperforms all baselines on the MSR block trace and Meta block trace, and is within 2% of the best (GDSF) for the Meta CDN trace. On the MSR block trace, the synthesized heuristic (Listing 3) blends raw access count with an EWMA-smoothed count, using a ghost multiplier to emphasize recently evicted objects. For the Meta block trace, it instead measures per-object access intensity and applies an additive penalty for ghost-list objects rather than a multiplicative one. For the 0.1% cache configuration, VULCAN outperforms all baselines on the Tencent object trace and is within 2% of the best on the MSR block and Twitter KV traces. On the Tencent object trace, VULCAN generates a GDSF-inspired heuristic similar to the MSR 10%

Listing 3. Simplified snippets of evolved heuristics for MSR and Meta block traces: actual heuristics are 29 and 50 lines of code.

```

1 // MSR block trace
2 freq    = 0.7*count + 0.3*EWMA(count, alpha=0.2)
3 size_pen = log2(size)
4 ghost_mult = 1 + 2*ghost_count
5 utility  = (last_access/1e6 + 10*freq - size_pen) *
             ghost_mult
6
7 // Meta block trace
8 intensity = count / (last_access - insert_time)
9 freq      = log2(count) * (10*intensity)
10 size_fac  = 1 / log2(size)
11 ghost_bonus = 1000 * (1 + log2(ghost_count))
12 utility   = (0.6*last_access + 400*freq) * size_fac +
             ghost_bonus

```

case, fine-tuning the constants for frequency and size to suit the Tencent workload and smaller cache.

Impact of removing listeners. Removing listeners slightly degrades performance: VULCAN-NoListener still outperforms all baselines on 7 instances but trails full VULCAN by 0.83% average MRR. Cache eviction heuristics are invoked at extremely high frequency, making complex listeners prohibitively expensive; as a result, most gains here stem from the RANK interface rather than derived features.

Quantifying the developer effort with VULCAN. The heuristic template required 133 lines of meaningful changes over a simple LRU baseline (excluding routine variable renaming), with no per-heuristic review needed beyond this one-time effort.

7.3 Memory Tiering

Memory tiering expands effective memory capacity by placing pages across fast local DRAM and slower tiers such as CXL-attached memory or NVM. The central policy problem is deciding *which pages* should occupy scarce DRAM: ideally, frequently accessed pages should be promoted to DRAM, while colder pages remain in slower tiers. We use VULCAN to synthesize page promotion heuristics and seek to answer whether they generalize from an emulated tiered-memory setup to real CXL hardware.

Manually designed heuristics. Existing tiering systems [56, 62, 80, 103] use sampled access information – from mechanisms such as Intel PEBS [43] or page-table accessed-bit scanning [74] – to estimate page hotness and migrate pages accordingly. Memtis [56] represents one class of designs: it promotes pages whose sampled access count exceeds a dynamically adjusted hotness threshold and demotes pages once their cooled access counts fall below it. ARMS [103] represents another class, using a ranking-based approach instead: it computes per-page scores from multi-timescale access averages, ranks pages by those scores, and migrates the highest-ranked pages to DRAM.

Template. We formulate the page migration problem as a RANK task in VULCAN, similar to ARMS [103]: where the goal is to compute the ‘hotness’ of a page and use this as a ranking function. The raw signals provided for this task include per-page access counts over a time window (of 500ms) and the observed bandwidths of the hot and cold tier. The developer-written template invokes libVulcan’s RANK periodically every 1 s.

Evaluator. We score candidate heuristics by running a single workload on a two-tier memory *emulator*. The emulator runs on a dual-socket Xeon Cloudlab [28] c220g5 node: the near tier is socket 0’s DRAM, and the far tier is socket 1’s DRAM with its uncore frequency – the clock governing the mesh interconnect, last-level cache, and memory controllers – clamped to the minimum ratio. This inflates remote-access latency to *emulate* that of a CXL-attached memory, while avoiding the cost of running heuristic search directly on scarce CXL hardware. After synthesis, we analyze whether the resulting heuristic generalizes to other workloads by testing it on our CXL-based testbed.

Evolution setup. We seed the search with a “promote nothing” heuristic that scores every page as 0 (*i.e.*, no migrations happen). We then run OpenEvolve [88] for 100 iterations, where heuristic candidates are scored by their end-to-end goodput on a specific application; the best heuristic at iteration 100 is carried forward to the generalization study below.

Generalization to real CXL hardware and unseen workloads. We take this single heuristic synthesized against a single application on an emulator and, without further tuning, evaluate it on a real CXL machine across a diverse set of workloads. Our experimental setup pairs a dual-socket Xeon host with a Micron CZ120 CXL 2.0 memory expander (256 GiB), connected to socket 0 over PCIe Gen5 x8 and exposed as a CPU-less NUMA node. The test workloads span three domains: GUPS [76] (memory microbenchmark used for training), GapBS Betweenness Centrality and PageRank [15] (graph analytics), and Silo TPCC [96] (in-memory database). All results are reported normalized to ARMS [103].

Results. Listing 4 shows the synthesized heuristic, VULCAN-GUPS, which used the GUPS benchmark [76] as the

Listing 4. Memory tiering heuristic synthesized by VULCAN.

```
1 config.add_listeners(accesses, {EWMA({0.95, 0.35})});
2 float score_fn(thor::feature_store fs, int64_t obj_id):
3     double fast = fs.get_ewma(accesses, obj_id, 0.95);
4     double slow = fs.get_ewma(accesses, obj_id, 0.35);
5     double accel = (slow > 0.001) ? (fast / slow) : 1.0;
6     return fast * (1.0 + 0.55 * min(accel, 2.5));
```

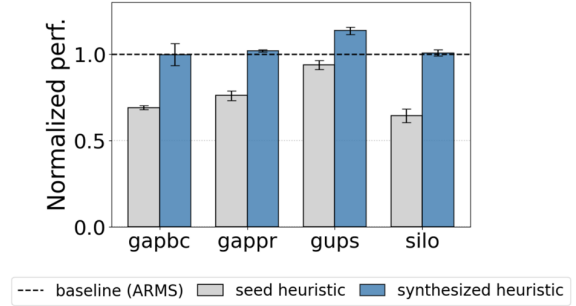


Figure 9. Performance of VULCAN-synthesized heuristic compared to the no-op seed heuristic

evaluator. This synthesized heuristic is tested on the CXL testbed, and the results of this are shown in Figure 9. From Listing 4, we observe that the heuristic uses the ratios of two EWMA’s to derive a metric called the *acceleration*, which it uses to make promotion decisions. We observe that this synthesized heuristic outperforms ARMS by 10.2% on GUPS on CXL, and matches the performance of ARMS on all other workloads, demonstrating how heuristics generated using an emulator-based evaluator generalize to real testbeds as well.

In Figure 10, we repeat this experiment, but using other workloads as the train set during evolution to get heuristics VULCAN-GapBC and VULCAN-Silo. We similarly test these heuristics on the CXL testbed and observe that they match or exceed the performance of ARMS on other workloads as well. The only exception to this trend is VULCAN-GapBC, which only achieves 0.96× the performance of ARMS on the train workload; we hypothesize that this could be because GapBS-BC is a workload that performs a breadth-first search over a graph, and hence, can result in random pointer chasing patterns that are not well captured by a single page promotion heuristic.

8 Related Work

Traditional learning-based policies. Prior ML-based systems policies, particularly neural policies [6, 53, 92, 104, 106, 114], have demonstrated that data-driven approaches can outperform fixed heuristics on specific instances. However, neural approaches introduce significant practical challenges: opaque behavior that complicates debugging [64], complex training and deployment pipelines [7, 31], inference overheads in the control path [31, 110], and safety concerns that limit adoption [84]. VULCAN takes a different stance: it avoids



Figure 10. Generalization of VULCAN synthesized heuristics

neural inference in the hot path entirely, confining learning to an offline search over small, interpretable LLM-generated code snippets.

AI-driven algorithm and heuristic design. Recent work has explored using LLMs to discover new heuristics via evolutionary search [70, 82]. One line of work modifies existing human-designed algorithms incrementally, such as automatic BBR variants [24, 40]. A complementary line focuses on search strategies: Glia [37] employs multiple specialized LLM agents, and Robusta [48] uses counterexamples to harden candidate heuristics. As we show in §2.2, unconstrained approaches of this kind can produce safety and integration bugs. These search strategies are complementary to VULCAN: any of them, combined with VULCAN’s RANK and VALUE interfaces and listeners, can yield similar benefits.

Domain-specific languages for execution safety. Restricted languages such as Rust and eBPF [32, 85] require complex verifiers and carry steep learning curves; even then, they do not enforce leak-freedom [32, 50], which is too restrictive a guarantee to provide for general programs. General-purpose verifiers [55, 57] can encode execution-safety properties as pre- and post-conditions, but require substantial per-program annotation effort, including loop invariants, termination arguments, and framing conditions [39, 69]. ANVIL takes the opposite extreme: by restricting LLM-written code to simple stateless functions implementing RANK or VALUE, it ensures P1–P3 by construction without any annotations.

9 Discussion and Conclusion

This paper presented VULCAN, a framework for synthesizing practical systems heuristics via LLMs. Our results suggest that the key challenge is not merely generating heuristics, but identifying the right synthesis boundaries: interfaces expressive enough to capture performant policies, yet constrained enough to preserve safety and efficient execution. VULCAN achieves this via RANK and VALUE interfaces for policy logic, listeners for state management, and ANVIL for execution-safety guarantees. VULCAN-generated heuristics match or outperform both human-designed and unconstrained search approaches.

Our experience also highlights the enduring role of human expertise: domain knowledge built over decades of research allows developers to identify meaningful runtime signals, separate core decision logic from mechanisms, and define operational constraints. Rather than replacing this expertise, VULCAN uses LLMs to explore and refine policy logic within the structure these insights impose.

We believe this work exposes a broader design space for LLM-friendly systems abstractions. While RANK and VALUE capture a broad class of resource-management heuristics, other tasks may require richer interfaces; e.g., coordination across policies or solving large optimizations. Similarly, the listener library can be extended to expose richer transformations over runtime state. Designing these abstractions jointly with policies remains an important direction for future work.

References

- [1] Horizontal pod autoscale. <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>. Accessed: Dec 2025.
- [2] Memcached - a distributed memory object caching system. <http://memcached.org/>. Accessed: 2026-04-22.
- [3] Redis - an in-memory data structure store, used as a database, cache, and message broker. <https://redis.io/>. Accessed: 2026-04-22.
- [4] Wikimedia cache dataset. https://wikitech.wikimedia.org/wiki/Data_Platform/Data_Lake/Traffic/Caching. Accessed: 2025-05-14.
- [5] Soheil Abbasloo, Yang Xu, and H. Jonathan Chao. C2tcp: A flexible cellular tcp to meet stringent delay requirements. *IEEE Journal on Selected Areas in Communications*, 37(4):918–932, 2019.
- [6] Soheil Abbasloo, Chen-Yu Yen, and H. Jonathan Chao. Classic meets modern: a pragmatic learning-based congestion control for the internet. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication*, SIGCOMM ’20, page 632–647, New York, NY, USA, 2020. Association for Computing Machinery.
- [7] Ibrahim Umit Akgun, Ali Selman Aydin, and Erez Zadok. Kmlib: Towards machine learning for operating systems. In *Proceedings of the On-Device Intelligence Workshop, co-located with the MLSys Conference*, pages 1–6, 2020.
- [8] Mohammad Alizadeh, Albert Greenberg, David A. Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murali Sridharan. Data center tcp (dctcp). In *Proceedings of the ACM SIGCOMM 2010 Conference*, SIGCOMM ’10, page 63–74, New York, NY, USA, 2010. Association for Computing Machinery.
- [9] Ami Tavory and Vladimir Dreizin and Benjamin Kosnik. *Policy-Based Data Structures*. GNU C++ Library, libstdc++, GNU Project. Accessed July 9, 2025.
- [10] Anthropic. Claude Code: Best practices for agentic coding, 2025. Accessed: 2026-04-08.
- [11] Anthropic. Introducing Claude Opus 4.5, November 2025. Accessed: 2026-04-08.
- [12] Venkat Arun and Hari Balakrishnan. Copa: Practical {Delay-Based} congestion control for the internet. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 329–342, Renton, WA, 2018. USENIX Association.
- [13] Nirav Atre, Justine Sherry, Weina Wang, and Daniel S Berger. Caching with delayed hits. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, pages 495–513, 2020.

- [14] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [15] Scott Beamer, Krste Asanović, and David Patterson. The gap benchmark suite. *arXiv preprint arXiv:1508.03619*, 2015.
- [16] Nathan Beckmann, Haoxian Chen, and Asaf Cidon. {LHD}: Improving cache hit rate by maximizing hit density. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 389–403, Renton, WA, 2018. USENIX Association.
- [17] Benjamin Berg, Daniel S Berger, Sara McAllister, Isaac Grosf, Sathya Gunasekar, Jimmy Lu, Michael Uhlar, Jim Carrig, Nathan Beckmann, Mor Harchol-Balter, et al. The {CacheLib} caching engine: Design and experiences at scale. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 753–768. USENIX Association, 2020.
- [18] Daniel S Berger, Benjamin Berg, Timothy Zhu, Siddhartha Sen, and Mor Harchol-Balter. {RobinHood}: Tail latency aware caching–dynamic reallocation from {Cache-Rich} to {Cache-Poor}. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 195–212, Carlsbad, CA, 2018. USENIX Association.
- [19] Zhuohang Bian, Feiyang Wu, Teng Ma, and Youwei Zhuo. Tokencake: A kv-cache-centric serving framework for llm-based multi-agent applications. *arXiv preprint arXiv:2510.18586*, 2025.
- [20] Aaron Blankstein, Siddhartha Sen, and Michael J Freedman. Hyperbolic caching: Flexible caching for web applications. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*, pages 499–511, 2017.
- [21] Neal Cardwell, Yuchung Cheng, C Stephen Gunn, Soheil Hassas Yeganeh, and Van Jacobson. Bbr: Congestion-based congestion control. *Communications of the ACM*, 60(2):58–66, 2017.
- [22] Jiayi Chen, Nihal Sharma, Tarannum Khan, Shu Liu, Brian Chang, Aditya Akella, Sanjay Shakkottai, and Ramesh K Sitaraman. Darwin: Flexible learning-based cdn caching. In *Proceedings of the ACM SIGCOMM 2023 Conference*, ACM SIGCOMM ’23, page 981–999, New York, NY, USA, 2023. Association for Computing Machinery.
- [23] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgan Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.
- [24] Audrey Cheng, Shu Liu, Melissa Pan, Zhifei Li, Shubham Agarwal, Mert Cemri, Bowen Wang, Alexander Krentsel, Tian Xia, Jongseok Park, et al. Let the barbarians in: How ai can accelerate systems performance research. *arXiv preprint arXiv:2512.14806*, 2025.
- [25] Ludmila Cherkasova. *Improving WWW proxies performance with greedy-dual-size-frequency caching policy*. Hewlett-Packard Laboratories, Palo Alto, CA, USA, 1998.
- [26] Mo Dong, Tong Meng, Doron Zarchy, Engin Arslan, Yossi Gilad, Brighton Godfrey, and Michael Schapira. {PCC} vivace: {Online-Learning} congestion control. In *15th USENIX symposium on networked systems design and implementation (NSDI 18)*, pages 343–356, 2018.
- [27] Thaleia Dimitra Doudali, Sergey Blagodurov, Abhinav Vishnu, Sudhanva Gurumurthi, and Ada Gavrilovska. Kleio: A hybrid memory page scheduler with machine intelligence. In *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing*, HPDC ’19, page 37–48, New York, NY, USA, 2019. Association for Computing Machinery.
- [28] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, et al. The design and operation of {CloudLab}. In *2019 USENIX annual technical conference (USENIX ATC 19)*, pages 1–14, 2019.
- [29] Rohit Dwivedula, Divyanshu Saxena, Aditya Akella, Swarat Chaudhuri, and Daehyeok Kim. Man-made heuristics are dead. long live code generators! In *Proceedings of the 24th ACM Workshop on Hot Topics in Networks*, pages 51–60, 2025.
- [30] Gil Einziger, Roy Friedman, and Ben Manes. Tynlfu: A highly efficient cache admission policy. *ACM Transactions on Storage (ToS)*, 13(4):1–31, 2017.
- [31] Henrique Fingler, Isha Tarte, Hangchen Yu, Ariel Szekely, Bodun Hu, Aditya Akella, and Christopher J. Rossbach. Towards a machine learning-assisted kernel with lake. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS 2023, page 846–861, New York, NY, USA, 2023. Association for Computing Machinery.
- [32] Bolaji Gbadamosi, Luigi Leonardi, Tobias Pulls, Toke Høiland-Jørgensen, Simone Ferlin-Reiter, Simo Sorce, and Anna Brunström. The ebpf runtime in the linux kernel. *arXiv preprint arXiv:2410.00026*, 2024.
- [33] P. Goyal, H.M. Vin, and Haichen Cheng. Start-time fair queueing: a scheduling algorithm for integrated services packet switching networks. *IEEE/ACM Transactions on Networking*, 5(5):690–704, 1997.
- [34] Robert Grandl, Mosharaf Chowdhury, Aditya Akella, and Ganesh Ananthanarayanan. Altruistic scheduling in Multi-Resource clusters. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 65–80, Savannah, GA, November 2016. USENIX Association.
- [35] Robert Grandl, Srikanth Kandula, Sriram Rao, Aditya Akella, and Janardhan Kulkarni. GRAPHENE: Packing and Dependency-Aware scheduling for Data-Parallel clusters. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 81–97, Savannah, GA, November 2016. USENIX Association.
- [36] Sangtae Ha, Injong Rhee, and Lisong Xu. Cubic: a new tcp-friendly high-speed tcp variant. *ACM SIGOPS operating systems review*, 42(5):64–74, 2008.
- [37] Pouya Hamadani, Pantea Karimi, Arash Nasr-Esfahany, Kimia Noorbakhsh, Joseph Chandler, Ali ParandehGheibi, Mohammad Alizadeh, and Hari Balakrishnan. Glia: A human-inspired ai for automated systems design and optimization. *arXiv preprint arXiv:2510.27176*, 2025.
- [38] Mingzhe Hao, Levent Toksoz, Nanqin Li, Edward Edberg Halim, Henry Hoffmann, and Haryadi S Gunawi. {LinnOS}: Predictability on unpredictable flash storage with a light neural network. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 173–190, 2020.
- [39] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R Lorch, Bryan Parno, Michael L Roberts, Srinath Setty, and Brian Zill. Ironfleet: proving practical distributed systems correct. In *Proceedings of the 25th Symposium on Operating Systems Principles*, pages 1–17, 2015.
- [40] Zhiyuan He, Aashish Gottipati, Lili Qiu, Yuqing Yang, and Francis Y Yan. Congestion control system optimization with large language models. *arXiv preprint arXiv:2508.16074*, 2025.
- [41] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*, 2021.

- [42] Qi Huang, Ken Birman, Robbert Van Renesse, Wyatt Lloyd, Sanjeev Kumar, and Harry C Li. An analysis of facebook photo caching. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, pages 167–181, 2013.
- [43] Intel Corporation. Intel 64 and ia-32 architectures software developer manuals. 2018.
- [44] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- [45] Wenzel Jakob. pybind11 documentation, 2024.
- [46] Theodore Johnson and Dennis Shasha. 2q: A low overhead high performance buffer management replacement algorithm. In *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB '94*, page 439–450, San Francisco, CA, USA, 1994. Morgan Kaufmann Publishers Inc.
- [47] Juncheng Yang (1a1a11a). libcachsim: a high performance library for building cache simulators. <https://github.com/1a1a11a/libCacheSim>, 2023. Accessed: 2025-06-28.
- [48] Pantea Karimi, Dany Rouhana, Pooria Namyar, Siva Kesava Reddy Kakarla, Venkat Arun, and Behnaz Arzani. Robust heuristic algorithm design with llms. *arXiv preprint arXiv:2510.08755*, 2025.
- [49] Mohammed F Kharram, Soohyeon Choi, Mohammad Alkhanafseh, and David Mohaisen. Security and quality in llm-generated code: A multi-language, multi-model analysis. *IEEE Transactions on Dependable and Secure Computing*, 2026.
- [50] Steve Klabnik, Carol Nichols, and Chris Krycho. Reference cycles can leak memory. In *The Rust Programming Language*, chapter 15.6. Rust 1.90.0, Edition 2024. Accessed: 2026-05-08.
- [51] Gautam Kumar, Nandita Dukkipati, Keon Jang, Hassan M. G. Wasel, Xian Wu, Behnam Montazeri, Yaogong Wang, Kevin Springborn, Christopher Alfeld, Michael Ryan, David Wetherall, and Amin Vahdat. Swift: Delay is simple and effective for congestion control in the datacenter. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication, SIGCOMM '20*, page 514–528, New York, NY, USA, 2020. Association for Computing Machinery.
- [52] Mayuresh Kunjir, Brandon Fain, Kamesh Munagala, and Shvinnath Babu. Robus: fair cache allocation for data-parallel workloads. In *Proceedings of the 2017 ACM International Conference on Management of Data*, pages 219–234, New York, NY, USA, 2017. Association for Computing Machinery.
- [53] Daniar H. Kurniawan, Rani Ayu Putri, Peiran Qin, Kahfi S. Zulkifli, Ray A. O. Sinurat, Janki Bhimani, Sandeep Madireddy, Achmad Imam Kistijantoro, and Haryadi S. Gunawi. Heimdall: Optimizing storage i/o admission with extensive machine learning pipeline. In *Proceedings of the Twentieth European Conference on Computer Systems, EuroSys '25*, page 1109–1125, New York, NY, USA, 2025. Association for Computing Machinery.
- [54] Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. Shinkaevolve: Towards open-ended and sample-efficient program evolution. *arXiv preprint arXiv:2509.19349*, 2025.
- [55] Andrea Lattuada, Travis Hance, Jay Bosamiya, Matthias Brun, Chan-Hee Cho, Hayley LeBlanc, Pranav Srinivasan, Reto Achermann, Tej Chajed, Chris Hawblitzel, et al. Verus: A practical foundation for systems verification. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 438–454, 2024.
- [56] Taehyung Lee, Sumit Kumar Monga, Changwoo Min, and Young Ik Eom. Memtis: Efficient memory tiering with dynamic page classification and page size determination. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.
- [57] K Rustan M Leino and Michał Moskal. Co-induction simply: Automatic co-inductive proofs in a program verifier. In *International Symposium on Formal Methods*, pages 382–398. Springer, 2014.
- [58] Jianheng Ling, Pratik Worah, Yawen Wang, Yunchuan Kong, Anshul Kapoor, Chunlei Wang, Clifford Stein, Diwakar Gupta, Jason Behmer, Logan A. Bush, Prakash Ramanan, Rajesh Kumar, Thomas Chestna, Yajing Liu, Ying Liu, Ye Zhao, Kathryn S. McKinley, Meeyoung Park, and Martin Maas. Lava: Lifetime-aware vm allocation with learned distributions and adaptation to mispredictions, 2025.
- [59] Shutian Luo, Huanle Xu, Chengzhi Lu, Kejiang Ye, Guoyao Xu, Liping Zhang, Yu Ding, Jian He, and Chengzhong Xu. Characterizing microservice dependency and performance: Alibaba trace analysis. In *Proceedings of the ACM Symposium on Cloud Computing, SoCC '21*, page 412–426, New York, NY, USA, 2021. Association for Computing Machinery.
- [60] Mark Mansi, Bijan Tabatabai, and Michael M. Swift. CBMM: Financial advice for kernel memory managers. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 593–608, Carlsbad, CA, July 2022. USENIX Association.
- [61] Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. Learning scheduling algorithms for data processing clusters. In *Proceedings of the ACM Special Interest Group on Data Communication, SIGCOMM '19*, page 270–288, New York, NY, USA, 2019. Association for Computing Machinery.
- [62] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit Kanaujia, and Prakash Chauhan. Tpp: Transparent page placement for cxl-enabled tiered-memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2023.
- [63] Nimrod Megiddo and Dharmendra S Modha. {ARC}: A {Self-Tuning}, low overhead replacement cache. In *2nd USENIX Conference on File and Storage Technologies (FAST 03)*, San Francisco, CA, USA, 2003. USENIX Association.
- [64] Zili Meng, Minhu Wang, Jiasong Bai, Mingwei Xu, Hongzi Mao, and Hongxin Hu. Interpreting deep learning-based networking systems. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication, SIGCOMM '20*, page 154–171, New York, NY, USA, 2020. Association for Computing Machinery.
- [65] Pierre Michaud. Best-offset hardware prefetching. In *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 469–480. IEEE, 2016.
- [66] Radhika Mittal, Vinh The Lam, Nandita Dukkipati, Emily Blem, Hassan Wassel, Monia Ghobadi, Amin Vahdat, Yaogong Wang, David Wetherall, and David Zats. Timely: Rtt-based congestion control for the datacenter. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication, SIGCOMM '15*, page 537–550, New York, NY, USA, 2015. Association for Computing Machinery.
- [67] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. Heterogeneity-Aware cluster scheduling policies for deep learning workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 481–498. USENIX Association, November 2020.
- [68] Dushyanth Narayanan, Austin Donnelly, and Antony Rowstron. Write off-loading: Practical power management for enterprise storage. *ACM Transactions on Storage (TOS)*, 4(3):1–23, 2008.
- [69] Luke Nelson, Helgi Sigurbjarnarson, Kaiyuan Zhang, Dylan Johnson, James Bornholt, Emina Torlak, and Xi Wang. Hyperkernel: Push-button verification of an os kernel. In *Proceedings of the 26th Symposium on Operating Systems Principles*, pages 252–269, 2017.
- [70] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav

- Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025.
- [71] Erik Nygren, Ramesh K Sitaraman, and Jennifer Sun. The akamai network: a platform for high-performance internet applications. *ACM SIGOPS Operating Systems Review*, 44(3):2–19, 2010.
- [72] Chandandeep Singh Pabla. Completely fair scheduler. *Linux Journal*, 2009(184):4, 2009.
- [73] Rangeet Pan, Ali Reza Ibrahimzadeh, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. Lost in translation: A study of bugs introduced by large language models while translating code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–13, 2024.
- [74] Seongjae Park, Yunjae Lee, and Heon Y Yeom. Profiling dynamic data access patterns with controlled overhead and quality. In *Proceedings of the 20th International Middleware Conference Industrial Track*, 2019.
- [75] Padmanabhan Pillai and Kang G Shin. Real-time dynamic voltage scaling for low-power embedded operating systems. In *Proceedings of the eighteenth ACM symposium on Operating systems principles*, pages 89–102, 2001.
- [76] Steven J Plimpton, Ron Brightwell, Courtenay Vaughan, Keith Underwood, and Mike Davis. A simple synchronous distributed-memory algorithm for the hpcc randomaccess benchmark. In *2006 IEEE International Conference on Cluster Computing*, pages 1–7. IEEE, 2006.
- [77] François Pottier and Yann Régis-Gianas. Menhir: An LR(1) parser generator for OCaml, 2024.
- [78] Sriram Ramabhadran and Joseph Pasquale. Stratified round robin: A low complexity packet scheduler with bandwidth fairness and bounded delay. In *Proceedings of the 2003 conference on applications, technologies, architectures, and protocols for computer communications*, pages 239–250, New York, NY, USA, 2003. Association for Computing Machinery.
- [79] Muhammad Shihab Rashid, Christian Bock, Yuan Zhuang, Alexander Buchholz, Tim Esler, Simon Valentin, Luca Franceschi, Martin Wistuba, Prabhu Teja Sivaprasad, Woo Jung Kim, et al. SWE-polybench: A multi-language benchmark for repository level evaluation of coding agents. *arXiv preprint arXiv:2504.08703*, 2025.
- [80] Amanda Raybuck, Tim Stamler, Wei Zhang, Mattan Erez, and Simon Peter. Hemem: Scalable tiered memory management for big data applications and real nvm. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, pages 392–407, 2021.
- [81] Liana V Rodriguez, Farzana Yusuf, Steven Lyons, Eysler Paz, Raju Rangaswami, Jason Liu, Ming Zhao, and Giri Narasimhan. Learning cache replacement with {CACHEUS}. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 341–354. USENIX Association, 2021.
- [82] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- [83] Marta Rycbczyńska. Introducing maple trees. LWN.net, February 2021. Accessed: 2026-03-22.
- [84] Divyanshu Saxena, Jiayi Chen, Sujay Yadalam, Yeonju Ro, Rohit Dwivedula, Eric H Campbell, Aditya Akella, Christopher J Rossbach, and Michael Swift. How i learned to stop worrying and love learned os policies. In *Proceedings of the 2025 Workshop on Hot Topics in Operating Systems*, pages 1–7, New York, NY, USA, 2025. Association for Computing Machinery.
- [85] Matan Shachnai, Harishankar Vishwanathan, Srinivas Narayana, and Santosh Nagarakatte. Fixing latent unsound abstract operators in the ebpf verifier of the linux kernel. In *International Static Analysis Symposium*, pages 386–406, Cham, 2024. Springer, Springer Nature Switzerland.
- [86] Mohammad Shahradd, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 205–218. USENIX Association, July 2020.
- [87] Erfan Sharafzadeh, Raymond Matson, Jean Tourrilhes, Puneet Sharma, and Soudeh Ghorbani. {Self-Clocked} {Round-Robin} packet scheduling. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 1437–1465, Philadelphia, PA, 2025. USENIX Association.
- [88] Asankhaya Sharma. Openevolve: An open-source evolutionary coding agent. <https://github.com/codelion/openevolve>, 2025. Accessed: 2025-12-10.
- [89] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.
- [90] Madhavapeddi Shreedhar and George Varghese. Efficient fair queueing using deficit round robin. In *Proceedings of the conference on Applications, technologies, architectures, and protocols for computer communication*, pages 231–242, 1995.
- [91] Ramneet Singh, Sathvik Joel, Abhav Mehrotra, Nalin Wadhwa, Ramakrishna Bairi, Aditya Kanade, and Nagarajan Natarajan. Code researcher: Deep research agent for large systems code and commit history. Technical Report MSR-TR-2025-34, Microsoft, June 2025.
- [92] Zhenyu Song, Daniel S Berger, Kai Li, and Wyatt Lloyd. Learning relaxed belady for content distribution network caching. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 529–544, Santa Clara, CA, 2020. USENIX Association.
- [93] Zhenyu Song, Kevin Chen, Nikhil Sarda, Deniz Altınbüken, Eugene Brevdo, Jimmy Coleman, Xiao Ju, Pawel Jurczyk, Richard Schooler, and Ramki Gummadi. {HALP}: Heuristic aided learned preference eviction policy for {YouTube} content delivery network. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 1149–1163, Boston, MA, 2023. USENIX Association.
- [94] Jovan Stojkovic, Chaojie Zhang, Inigo Goiri, Esha Choukse, Haoran Qiu, Rodrigo Fonseca, Josep Torrellas, and Ricardo Bianchini. Tapas: Thermal- and power-aware scheduling for llm inference in cloud platforms. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS '25*, page 1266–1281, New York, NY, USA, 2025. Association for Computing Machinery.
- [95] Jovan Stojkovic, Chaojie Zhang, Inigo Goiri, Josep Torrellas, and Esha Choukse. Dynamollm: Designing llm inference clusters for performance and energy efficiency. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 1348–1362, 2025.
- [96] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, and Samuel Madden. Speedy transactions in multicore in-memory databases. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, pages 18–32, 2013.
- [97] Giuseppe Vietri, Liana V. Rodriguez, Wendy A. Martinez, Steven Lyons, Jason Liu, Raju Rangaswami, Ming Zhao, and Giri Narasimhan. Driving cache replacement with ML-based LeCaR. In *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 18)*, Boston, MA, July 2018. USENIX Association.
- [98] Carl A Waldspurger, Nohhyun Park, Alexander Garthwaite, and Irfan Ahmad. Efficient {MRC} construction with {SHARDS}. In *13th USENIX Conference on File and Storage Technologies (FAST 15)*, pages 95–110. USENIX Association, 2015.

- [99] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. In *Forty-first International Conference on Machine Learning*, 2024.
- [100] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [101] Zhanghao Wu, Wei-Lin Chiang, Ziming Mao, Zongheng Yang, Eric Friedman, Scott Shenker, and Ion Stoica. Can’t be late: Optimizing spot instance savings under deadlines. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 185–203, 2024.
- [102] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, Fan Yang, and Lidong Zhou. Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 595–610, Carlsbad, CA, October 2018. USENIX Association.
- [103] Sujay Yadalam, Konstantinos Kanellis, Michael Swift, and Shivaram Venkataraman. Arms: Adaptive and robust memory tiering system. *arXiv preprint arXiv:2508.04417*, 2025.
- [104] Chenxi Yang, Divyanshu Saxena, Rohit Dwivedula, Kshiteej Mahajan, Swarat Chaudhuri, and Aditya Akella. C3: Learning congestion controllers with formal certificates. *arXiv preprint arXiv:2412.10915*, 2024.
- [105] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- [106] Juncheng Yang, Ziming Mao, Yao Yue, and K. V. Rashmi. GL-Cache: Group-level learning for efficient and high-performance caching. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*, pages 115–134, Santa Clara, CA, February 2023. USENIX Association.
- [107] Juncheng Yang, Yao Yue, and K. V. Rashmi. A large scale analysis of hundreds of in-memory cache clusters at twitter. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 191–208. USENIX Association, November 2020.
- [108] Juncheng Yang, Yazhuo Zhang, Ziyue Qiu, Yao Yue, and Rashmi Vinayak. Fifo queues are all you need for cache eviction. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 130–149, New York, NY, USA, 2023. Association for Computing Machinery.
- [109] Tzu-Wei Yang, Seth Pollen, Mustafa Uysal, Arif Merchant, and Homer Wolfmeister. {CacheSack}: Admission optimization for google datacenter flash caches. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 1021–1036, Carlsbad, CA, 2022. USENIX Association.
- [110] Junxue Zhang, Chaoliang Zeng, Hong Zhang, Shuihai Hu, and Kai Chen. Liteflow: towards high-performance adaptive neural networks for kernel datapath. In *Proceedings of the ACM SIGCOMM 2022 Conference, SIGCOMM ’22*, page 414–427, New York, NY, USA, 2022. Association for Computing Machinery.
- [111] Yazhuo Zhang, Juncheng Yang, Yao Yue, Ymir Vigfusson, and K.V. Rashmi. SIEVE is simpler than LRU: an efficient Turn-Key eviction algorithm for web caches. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 1229–1246, Santa Clara, CA, April 2024. USENIX Association.
- [112] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning acting and planning in language models. *arXiv preprint arXiv:2310.04406*, 2023.
- [113] Ke Zhou, Si Sun, Hua Wang, Ping Huang, Xubin He, Rui Lan, Wenyan Li, Wenjie Liu, and Tianming Yang. Demystifying cache policies for photo stores at scale: A tencent case study. In *Proceedings of the 2018 International Conference on Supercomputing, ICS ’18*, page 284–294, New York, NY, USA, 2018. Association for Computing Machinery.
- [114] Wenbin Zhou, Zhixiong Niu, Yongqiang Xiong, Juan Fang, and Qian Wang. 3l-cache: Low overhead and precise learning-based eviction policy for caches. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies*, pages 237–254, Santa Clara, CA, 2025. USENIX Association.
- [115] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widayarsi, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.

A Challenges with LLM-based heuristic generation

To evaluate whether current LLMs can synthesize effective systems heuristics that meet these requirements (Table 3), we used a state-of-the-art model, Claude Opus 4.5, to generate caching heuristics in libCacheSim [47].

libCacheSim [47] is an event-driven cache simulator, in which new heuristics are defined by implementing event handlers for cache operations such as accesses, evictions, and insertions – each of which has a specific function signature detailed in the documentation (a systems integration requirement). Additionally, the libCacheSim engine also provides access to commonly used features – such as total object counts, the current timestamp, and the number of hits/misses – via raw pointers, which heuristics are allowed to read but must not mutate (another systems integration requirement).

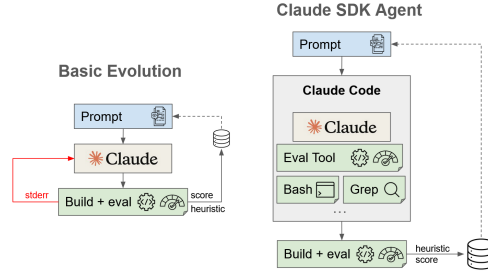


Figure 11. Evolutionary search with a simple agent vs. Claude Code. The dotted line represents the start of a new iteration; every iteration is seeded with samples of the best-performing programs so far to help guide to search.

A.1 Attempt 1: A simple evolutionary loop

We implement a evolutionary loop using a simple LLM agent (inspired by prior work such as [29, 82]), as shown in the left half of Figure 11. The agent is provided a prompt describing libCacheSim and its interfaces, which it uses to generate a candidate heuristic. The generated heuristic is then built and evaluated on a suite of ten traces from the CloudPhysics dataset [98]. If a build or runtime error occurs, the agent is provided the `stderr` to iteratively refine and correct its code. Once the heuristic compiles and executes successfully, both the implementation and its performance metrics (*i.e.*, miss rates) are stored in a database. Each such iteration constitutes the *generation* of a single heuristic. This process is repeated for fifty iterations, with higher-performing heuristics sampled from the database and incorporated into subsequent prompts to guide the search toward better designs.

Manual inspection revealed a recurring issue across all fifty heuristics. In libCacheSim, heuristics must report their per-object metadata footprint (in bytes) via a system variable used for memory accounting. Despite introducing additional per-object state, the LLM consistently under-reported this value. As a result, these heuristics exceeded their memory budget, yielding unfair comparisons against baselines that correctly account for memory usage and violating interface compliance (systems integration bug).

Despite these issues, after fifty iterations, the evolutionary process produced heuristics that “achieved” a 3%–5.1% improvement over FIFO according to the evaluator. However, these gains were partly driven by exploiting gaps in the evaluation harness (*i.e.*, under-reporting metadata usage). Moreover, heuristics with memory safety bugs also passed without crashing, as the harness exercised only limited execution paths. This shows that heuristics which compile, run successfully, and perform well on the evaluator may still violate the desired properties in Table 3.

A.2 Attempt 2: Agentic code synthesis

After initial experiments with simple agents showed that LLM synthesized code often violates desired properties, we asked whether these issues persist in more capable agentic frameworks that augment LLMs with external tools. To investigate, we repeated the heuristic synthesis experiment using Claude Code – a widely used coding agent that equips an LLM with capabilities such as file manipulation, shell execution and search. Such agents may better avoid interface-compliance bugs observed earlier, as they can spawn sub-agents to reason and understand the internals of systems they interact with more effectively.

To test this hypothesis, we repeated the experiment using the Claude Code Agent SDK (Claude Opus 4.5) to synthesize each candidate heuristic, as shown in the right half of Figure 11. In addition to default tools (bash, search, read, etc.), we provided a custom evaluate tool that builds the heuristic, runs it on CloudPhysics traces, and returns a JSON report of its performance relative to FIFO. Although the agent could execute these steps via bash, exposing them as a dedicated tool simplifies its workflow.

Using this more capable agentic setup, we run the evolutionary process for ten iterations, producing ten heuristics. As in Attempt 1, each iteration generates a candidate via a loop of proposing, evaluating, and refining; however, unlike the earlier setup – where the loop was limited to handling compiler and runtime errors – each iteration here is a richer multi-turn interaction. The agent can explore the codebase, reason about interfaces, modify implementations, and invoke tools such as evaluate multiple times before producing a candidate.

Generating each candidate heuristic with Claude Code took 11 – 28 minutes, consumed ~2M tokens (~\$3.20 via AWS Bedrock), and involved an average of 57.4 tool calls. The resulting heuristics achieved 36.4% – 46.8% improvement over FIFO, as scored by the evaluator: significantly outperforming those found by the simpler agent. They also exhibited diverse strategies: the best-performing heuristic uses a prioritization function combining frequency tiers, size-cost normalization, and recency protection, while the second-best applies a concave transformation (a sqrt-log blend) to weight access frequency.

Unlike the previous setup – where even modest gains were partly driven by reward hacking (*i.e.*, under-reporting metadata to bypass memory accounting) – the heuristics produced by this agentic framework did not exploit that loophole. However, several still violated interface assumptions: for instance, several heuristics – including the best-performing one – overwrote `next_access_vtime` (per-object state maintained by `libCacheSim`) to store the current time, violating the system interface by treating a read-only field as writable (a systems integration bug). Additionally, a double-free bug (an execution safety bug) in the second-best heuristic went undetected by the evaluator; `libCacheSim` exposes two removal paths – `evict()`, which selects and removes objects, and `remove()`, which deletes objects by ID. Since our evaluator exercised only the `evict()` path, the bug in `remove` passed through evaluation unnoticed since it was never triggered.

A.3 Summary

This case study reinforces a growing body of evidence that LLMs routinely introduce bugs and safety violations when generating complex code [44, 49, 73, 115] – and that more capable agents are no exception, frequently requiring humans in the loop to produce reliable output [79, 91].

While LLMs may be more successful at synthesizing code for self-contained coding tasks [14, 23, 41], systems heuristics are usually more complex: they may maintain internal state in complex data structures [83], involve intricate state-transition logic [36, 63, 108], or might be dispersed across multiple files and control paths [60]. Allowing an LLM to synthesize or modify such heuristics end-to-end – reasoning simultaneously about policy logic, state management, and mechanisms – widens the attack surface for subtle bugs and property violations in the synthesized code.

B RANK-based Cache Eviction Heuristics

The natural language prompt used for the cache eviction case study (§7.2) is shown in Listing 1.

Listing 1: Prompt used for cache eviction heuristic synthesis

Context. You are a systems researcher designing a new cache eviction policy. Whenever the cache is full and a new object must be inserted, this policy is invoked to select which cached object to evict. The policy has access to per-object features such as object size, insertion time, last access time, and access count, along with global information about recently evicted objects.

Policy Model. The policy is implemented using *thor*, a framework that separates *policy* from *mechanism*. Your task is to implement a *scoring function* that returns the **utility** of an object:

Higher utility $\Rightarrow$ more valuable to keep

When eviction is required, the object with the *lowest utility* is selected.

Lazy Priority Queue Execution Model. The scoring function is evaluated lazily:

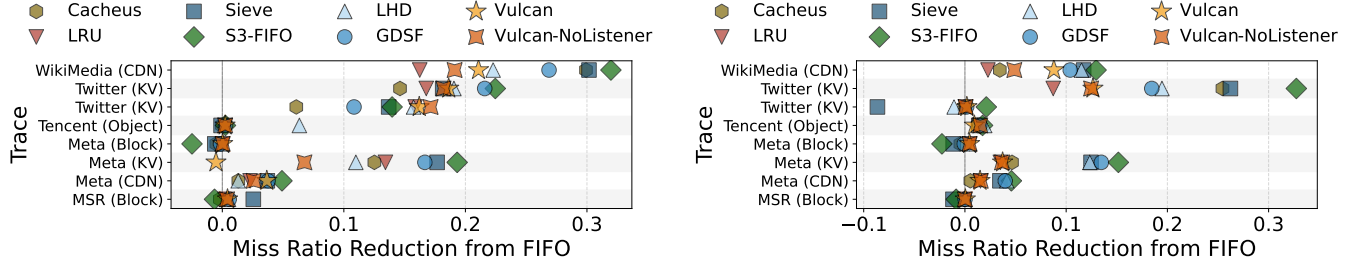
- A score is recomputed only when an object is inserted or accessed.
- The resulting score is cached in the priority queue.
- Idle objects are *not* rescored over time.

Therefore, expressions such as `curr_time - last_access` are ineffective because, at rescore time, `curr_time  $\approx$  last_access`. Instead, use:

- `last_access` directly for recency,
- `insertion_time` directly for age ordering,
- or other stable per-object signals.

Global Features.

- `f_curr_time (i64)`: Current logical time.



(a) Large cache size (10%), objects of same sizes

(b) Small cache size (0.1%), objects of same sizes

Figure 12. Performance of instance-specialized heuristics versus baselines for two classes of instances.

- `f_ghost` (i64): IDs of recently evicted objects.

Per-Object Features.

- `f_size` (i64): Object size in bytes.
- `f_insertion_time` (i64): Insertion timestamp.
- `f_last_access` (i64): Most recent access timestamp.
- `f_count` (i64): Access count since insertion.

Listeners. Listeners determine how feature values are stored and queried. Examples include:

- `RollingWindow(N)`
- `EWMA({alpha})`
- `RollingPercentile(N)`
- `RollingCount(N)`

Example listener configuration:

```
config.add_listeners(
    f_last_access, {thor::listeners::object::RollingWindow(5)}
);

config.add_listeners(
    f_curr_time, {thor::listeners::global::RollingWindow(1)}
);
```

Expected Output Format. The generated file must contain:

1. Listener configuration
2. Scoring function

Example Scoring Function.

```
auto scoring_fn =
 [&](const thor::feature_store& fs, int64_t obj_id) -> double {
    double last_access = fs.get_latest(f_last_access, obj_id);
    double count = fs.get_latest(f_count, obj_id);
    double ghost_count = fs.get_count(f_ghost, obj_id);
    double rel_freq = count;
    double ghost_penalty = ghost_count * 1e6;
    return last_access + rel_freq * 10.0 - ghost_penalty;
};
```

B.1 Full Results

Figure 12 shows the performance of VULCAN-generated heuristics for two classes of instances where all objects in the cache have the same size. As presented in Section 7.2, we tested these heuristics against seven baselines and find that VULCAN-generated heuristics are generally competitive against strong manually designed baselines.